

Universidad Nacional de Colombia

Proyecto Final

Permisos bajo control: roles seguros para una aplicación interna mediante Backtracking

Asignatura: Introducción a las Ciencias de la Computación

Integrantes del grupo:

Juliana Bedoya Ramírez
Laura Sofía Noreña Campiño
Andrea Tangarife Arias

Docente:

Carlos Manuel Orrego Franco

Manizales

19 de junio de 2026

Resumen breve

Este proyecto consiste en resolver un problema de asignación de roles y permisos dentro de una aplicación interna utilizando el algoritmo de Backtracking.

El objetivo es encontrar una asignación adecuada de roles para diferentes usuarios, teniendo en cuenta varias reglas de seguridad que evitan combinaciones de permisos incorrectas.

Para desarrollar la solución se realizó un modelo del problema y una implementación en Python, donde el algoritmo prueba diferentes posibilidades, descarta las que no cumplen las restricciones y encuentra una asignación válida cuando existe.

En caso de que las reglas impidan una solución, el programa indica que no existe una solución posible.

Introducción y motivación

En una aplicación es importante controlar qué puede hacer cada usuario, porque no todas las personas deben tener los mismos permisos.

Una mala asignación de roles puede causar problemas de seguridad. Por ejemplo, una persona no debería poder crear una solicitud, aprobarla y después editarla, porque estaría controlando todo el proceso.

Por eso decidimos trabajar este problema, donde debemos asignar roles a diferentes usuarios teniendo en cuenta varias reglas.

Usamos Backtracking porque este algoritmo permite revisar diferentes posibilidades y descartar las que no cumplen con las condiciones del problema.

Con este proyecto buscamos entender cómo un algoritmo visto en clase puede aplicarse a un caso relacionado con la ciberseguridad.

Descripción del problema

El proyecto se basa en una situación de una empresa llamada *Finanzas Claras Manizales*, que necesita una aplicación interna para manejar solicitudes, aprobaciones y auditorías.

El problema aparece porque una persona podría tener demasiados permisos y realizar varias funciones importantes al mismo tiempo. Esto puede afectar la seguridad del sistema.

La solución propuesta consiste en asignar roles a usuarios ficticios y verificar que la asignación cumpla las reglas establecidas.

Usuarios

- Ana
- Luis
- María
- Camilo
- Sofía

Roles

- Lector
- Editor
- Aprobador
- Auditor
- Administrador

Reglas de seguridad

- Debe existir al menos un auditor.
- Debe existir al menos un aprobador.
- Una persona no puede ser aprobador y auditor al mismo tiempo.
- Solo una persona puede ser administrador.

- Camilo no puede ser administrador.
- Sofía solo puede ser lectora o auditora.

El objetivo es que el algoritmo encuentre una asignación válida o indique que no existe una solución posible.

Modelamiento con Backtracking

Podemos usar el Backtracking como solución porque el estado del problema es una asignación parcial de roles a los usuarios.

El estado inicial es un diccionario vacío. En cada paso seleccionamos un usuario y probamos todos los roles posibles.

La función de factibilidad verifica las restricciones de seguridad. Si una asignación viola una regla, el algoritmo realiza un retroceso y busca una alternativa diferente.

Además, podemos utilizar poda para evitar explorar opciones que ya sabemos que son inválidas.

El objetivo es encontrar una asignación completa donde todos los usuarios tengan un rol y se cumplan todas las reglas del sistema.

Ejemplo manual pequeño

Antes de dar solución por completo al problema, decidimos analizar una versión simplificada para evidenciar cómo se utiliza el algoritmo de Backtracking, cómo explora las posibles soluciones y cómo puede aplicar poda para mejorar la eficiencia.

```
1 usuarios = ["Ana", "Luis"]
2 roles = ["Lector", "Administrador"]
3
4 for rol_ana in roles:
5     for rol_luis in roles:
6
7         administradores = 0
8
9         if rol_ana == "Administrador":
10             administradores += 1
11
12         if rol_luis == "Administrador":
13             administradores += 1
14
15         if administradores <= 1:
16
17             print("Solución válida:")
18             print("Ana ->", rol_ana)
19             print("Luis ->", rol_luis)
20             print()

```

Solución válida:
Ana -> Lector
Luis -> Lector

Solución válida:
Ana -> Lector
Luis -> Administrador

Solución válida:
Ana -> Administrador

Figura 1: Representación simplificada del proceso de Backtracking.

Decisiones del grupo

¿Qué representación eligieron y por qué?

Elegimos usar listas porque nos permitían organizar fácilmente los usuarios, roles y permisos, además, se podía recorrer la información de manera sencilla.

¿Qué restricciones implementaron primero?

Primero, verificamos que un mismo rol no se asignara dos veces al mismo usuario, y que cada usuario tuviera su rol en base a las restricciones propuestas.

¿Qué bug encontraron durante el desarrollo?

Al principio, algunos usuarios aparecían con roles repetidos porque no estábamos llevando a cabo correctamente las asignaciones.

¿Qué caso de prueba falló inicialmente?

Falló el caso en el que varios usuarios intentaban tener el mismo rol cuando este solo podía asignarse una vez.

¿Qué poda implementaron?

Eliminábamos inmediatamente las asignaciones que incumplían alguna restricción, sin seguir evaluando más posibilidades.

¿Qué cambiarían si tuvieran más tiempo?

Mejoraríamos mucho más el código base del programa, podríamos utilizar herramientas más sofisticadas con una investigación profunda y agregaríamos recursos para manejar casos especiales.

Implementación en Python

Para realizar este ejercicio decidimos desarrollar la programación en Google Colab, ya que es una herramienta que conocemos por el uso que le hemos dado anteriormente.

Por otro lado, la solución fue implementada usando el algoritmo de backtracking. Con este logramos asignar usuarios, roles y las restricciones necesarias para el problema.

Podemos observar que, gracias al tipo de algoritmo que usamos, los roles se asignan a los usuarios de una forma progresiva.

Esto significa que el algoritmo verifica en cada paso que las reglas propuestas se cumplan para poder continuar. Si detecta que una o varias reglas no se cumplen, prueba otras opciones y descarta de forma eficiente aquellas que infringen las restricciones.

Gracias a esto, existen múltiples formas de encontrar una solución eficiente.

```
32         return False
33
34     # 6. No permitir que se termine una asignación sin aprobador
35     # (solo se aplica cuando ya están todos los usuarios asignados)
36     if len(asignacion) == 4:
37         roles_finales = list(asignacion.values()) + [rol]
38
39         if "aprobador" not in roles_finales:
40             return False
41
42     return True
```

Solución completa

```
1 def es_solucion_completa(asignacion):
2
3     if len(asignacion) != len(usuarios):
4         return False
5
6     roles_asignados = asignacion.values()
7
8     if "auditor" not in roles_asignados:
9         return False
10
11     if "aprobador" not in roles_asignados:
12         return False
13
```

Figura 2: Implementación del algoritmo de Backtracking en Python.

Pruebas y resultados

Para verificar el funcionamiento del programa realizamos varias pruebas.

En una de ellas observamos cómo la asignación cumple todos los requisitos del problema, ya que cada rol está distribuido respetando todas las reglas definidas.

	Usuario	Rol
0	Ana	administrador
1	Luis	editor
2	Maria	editor
3	Camilo	aprobador
4	Sofia	auditor

Figura 3: Caso de prueba con solución válida.

También realizamos una prueba de un caso sin solución, agregando una regla contradictoria.

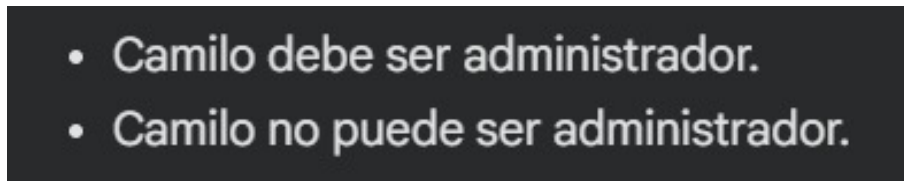
- 
- Camilo debe ser administrador.
 - Camilo no puede ser administrador.

Figura 4: Caso con restricciones contradictorias.

Ya que estas dos reglas no pueden cumplirse simultáneamente, el algoritmo exploró las diferentes posibilidades y determinó correctamente que no existe una solución válida para ese caso.

Estos resultados muestran que la implementación es capaz de encontrar caminos alternativos correctos y también detectar restricciones imposibles.

Conclusiones

Gracias a este trabajo pudimos observar que el algoritmo de Backtracking es una funcion util ya que este es capaz de resolver problemas donde existen muchas restricciones las cuales se deben de cumplir al mismo tiempo, ademas es una herramienta que gracias a la constante verificacion de las reglas muestra soluciones que sean validas cuando no se cumple una regla.

Por otro lado, el uso de Python ayudó en la implementación de la solución, facilitó las pruebas realizadas y permitió visualizar de manera clara los resultados obtenidos.

Finalmente, este proyecto nos ayudó a comprender mejor el funcionamiento del algoritmo de Backtracking, cómo se aplica en la resolución de problemas y en qué situaciones puede utilizarse para encontrar soluciones de forma eficiente.

Enlace al código

https://colab.research.google.com/drive/11fiQQFA0HdLR0w_xUBSint5zkIu-2K_A?usp=sharing